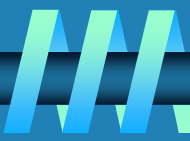
Dynamic Programming



D*ynamic Programming* is a general algorithm design technique for solving problems defined by recurrences with overlapping subproblems

- **Invented by American mathematician Richard Bellman in the 1950s to solve optimization problems and later assimilated by CS**

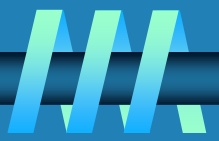
- **"Programming" here means "planning"**

- **Main idea:**
  - **set up a recurrence relating a solution to a larger instance to solutions of some smaller instances**
  - **solve smaller instances once**
  - **record solutions in a table**
  - **extract solution to the initial instance from that table**



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

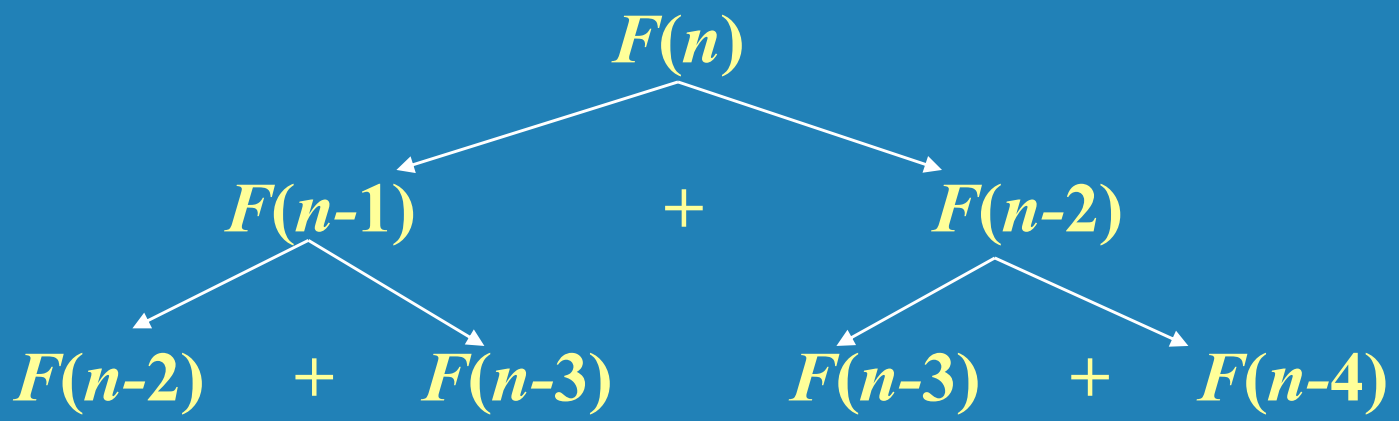
Example 1: Fibonacci numbers

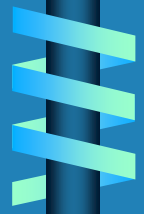


- **Recall definition of Fibonacci numbers:**

$$F(n) = F(n\text{-}1) + F(n\text{-}2)$$
$$F(0) = 0$$
$$F(1) = 1$$

- **Computing the nth Fibonacci number recursively (top-down):**





A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example 1: Fibonacci numbers (cont.)

Computing the nth Fibonacci number using bottom-up iteration and recording results:

$F(0) = 0$
$F(1) = 1$
$F(2) = 1+0 = 1$
…
$F(n\text{-}2) =$
$F(n\text{-}1) =$
$F(n) = F(n\text{-}1) + F(n\text{-}2)$

0	1	1	. . .	$F(n\text{-}2)$	$F(n\text{-}1)$	$F(n)$

Efficiency:
- **time**
- **space**

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

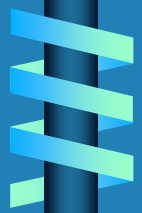
Example 2: Coin-row problem



There is a row of n coins whose values are some positive integers $c_1, c_2,...,c_n$, not necessarily distinct. The goal is to pick up the maximum amount of money subject to the constraint that no two coins adjacent in the initial row can be picked up.

E.g.: 5, 1, 2, 10, 6, 2. What is the best selection? 5+10+2 = 17

F(n) = # coins from the beginning to the nth integer.



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

DP solution to the coin-row problem

Let F(n) be the maximum amount that can be picked up from the row of n coins. To derive a recurrence for F(n), we partition all the allowed coin selections into two groups:

those without last coin – the max amount is ? $F(n) = F(n-1)$

those with the last coin -- the max amount is ? $F(n) = F(n-2) + c_n$

Thus we have the following recurrence

$$F(n) = \max\{c_n + F(n-2),\ F(n-1)\} \quad \text{for } n > 1,$$

$$F(0) = 0,\ F(1) = c_1$$

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

DP solution to the coin-row problem (cont.)

$$F(n) = \max\{c_n + F(n\text{-}2),\ F(n\text{-}1)\} \quad \text{for } n > 1,$$

$$F(0) = 0,\ F(1) = c_1$$

index	0	1	2	3	4	5	6
coins	--	5	1	2	10	6	2
F( )	0	5	5	7	15	15	17

c_1

$F(2) = \max\{5, 0+1\}$ ✗

$F(3) = \max\{5, 5+2\}$

$F(4) = \max\{7, 5+10\}$ c_4

$F(5) = \max\{15, 7+6\}$

$F(6) = \max\{15, 15+2\}$

c_6

Max amount:

Coins of optimal solution: go backwards c_6, c_4, c_1

Time efficiency: $\Theta(n)$

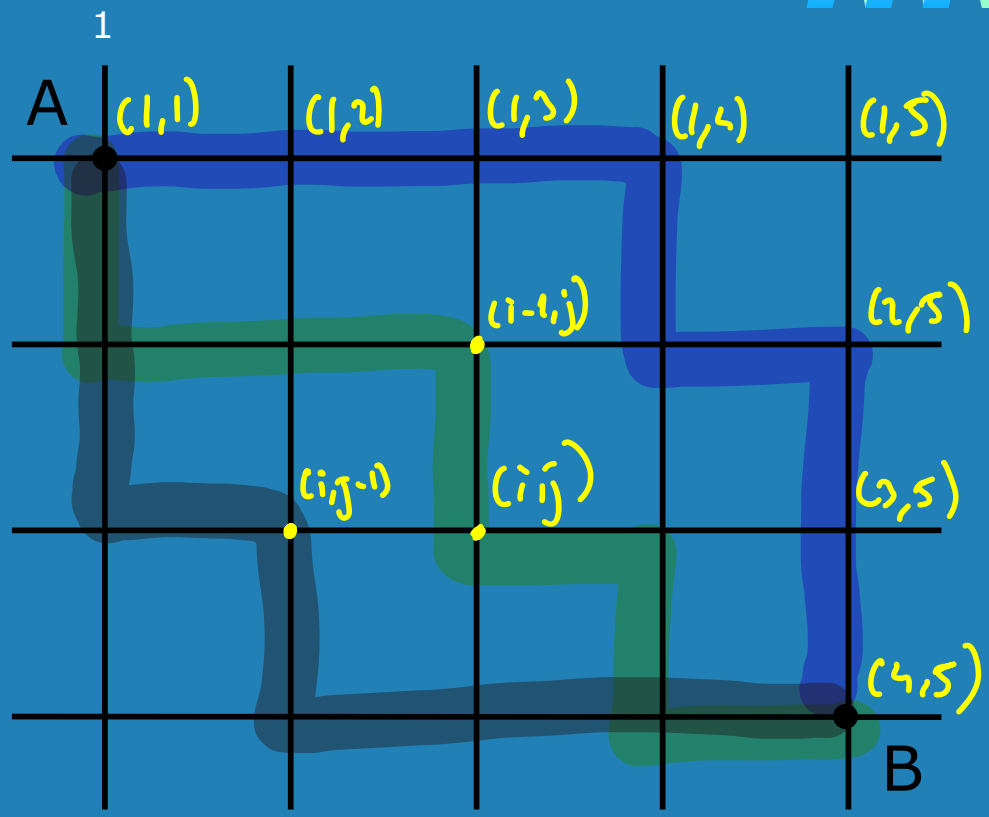
Space efficiency: $\Theta(n)$

Note: All smaller instances were solved.

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example 3: Path counting

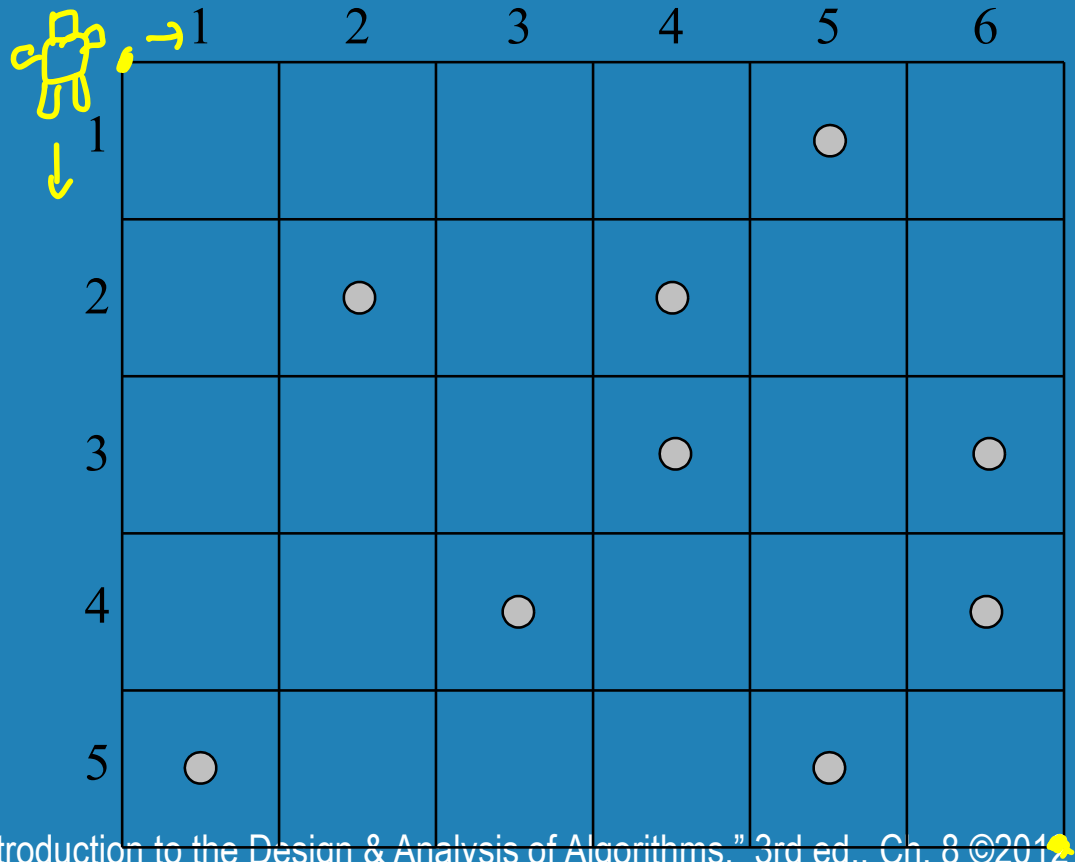
Consider the problem of counting the number of shortest paths from point A to point B in a city with perfectly horizontal streets and vertical avenues



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example 4: Coin-collecting by robot

Several coins are placed in cells of an $n \times m$ board. A robot, located in the upper left cell of the board, needs to collect as many of the coins as possible and bring them to the bottom right cell. On each step, the robot can move either one cell to the right or one cell down from its current location.



$c_{i,j} = 1$ if there is a coin in (i,j)

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Solution to the coin-collecting problem

Let F(i,j) be the largest number of coins the robot can collect and bring to cell (i,j) in the ith row and jth column.

The largest number of coins that can be brought to cell (i,j):

from the left neighbor ?

from the neighbor above?

The recurrence:

$$F(i, j) = \max\{F(i\text{-}1, j),\ F(i, j\text{-}1)\} + c_{ij} \quad \text{for } 1 \le i \le n,\ 1 \le j \le m$$

where $c_{ij} = 1$ if there is a coin in cell (i,j), and $c_{ij} = 0$ otherwise

$$F(0, j) = 0 \text{ for } 1 \le j \le m \quad \text{and } F(i, 0) = 0 \text{ for } 1 \le i \le n.$$

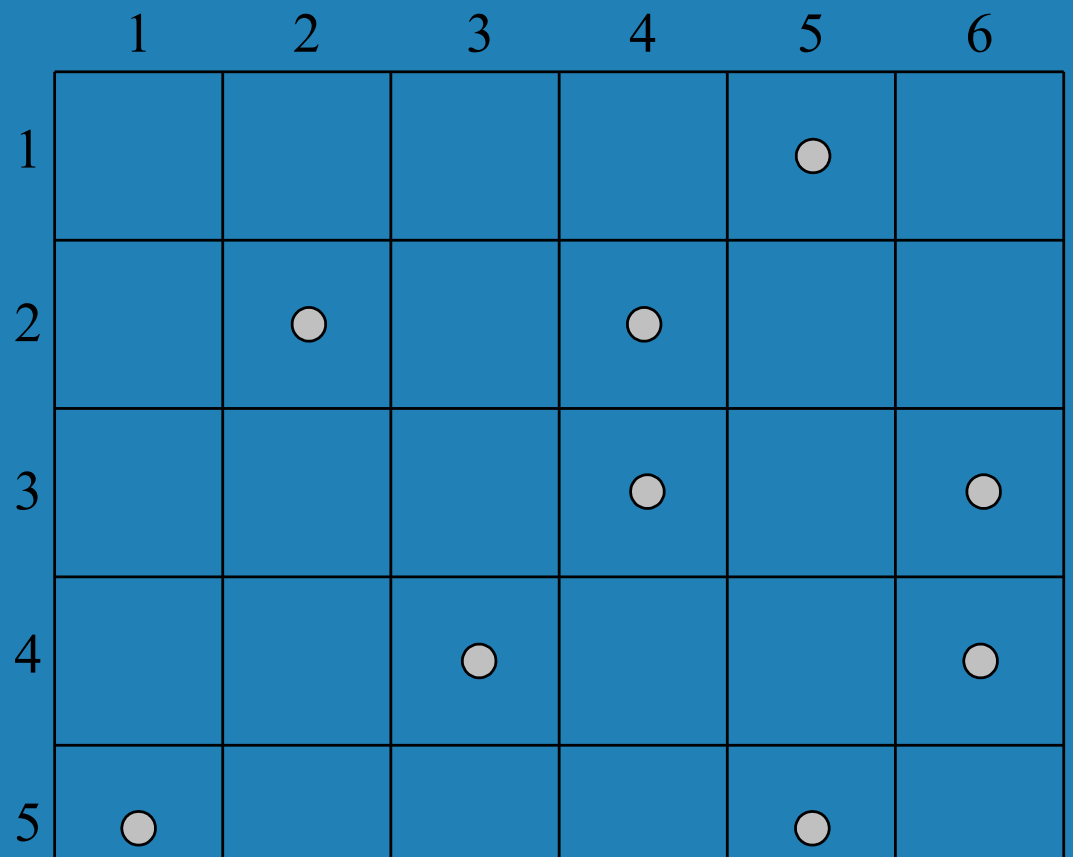
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Solution to the coin-collecting problem (cont.)

$F(i, j) = \max\{F(i\text{-}1, j),\ F(i, j\text{-}1)\} + c_{ij}$ for $1 \le i \le n$, $1 \le j \le m$

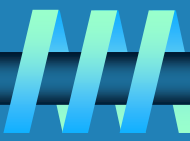
where $c_{ij} = 1$ if there is a coin in cell (i,j), and $c_{ij} = 0$ otherwise

$F(0, j) = 0$ for $1 \le j \le m$ and $F(i, 0) = 0$ for $1 \le i \le n$.



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Other examples of DP algorithms

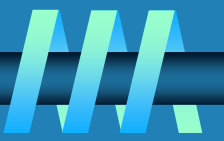


- Computing a binomial coefficient (# 9, Exercises 8.1)
- General case of the change making problem (Sec. 8.1)
- Some difficult discrete optimization problems:
  - knapsack (Sec. 8.2)
  - traveling salesman
- Constructing an optimal binary search tree (Sec. 8.3)
- Warshall's algorithm for transitive closure (Sec. 8.4)
- Floyd's algorithm for all-pairs shortest paths (Sec. 8.4)



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Knapsack Problem by DP



Given n items of

integer weights: $w_1 \quad w_2 \quad \dots \quad w_n$

values: $v_1 \quad v_2 \quad \dots \quad v_n$

a knapsack of integer capacity W

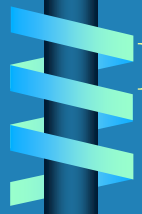
find most valuable subset of the items that fit into the knapsack

Consider instance defined by first i items and capacity j ($j \le W$).

Let $V[i,j]$ be optimal value of such instance. Then

$$V[i,j] = \begin{cases} \max \{V[i-1,j],\ v_i + V[i-1,j-w_i]\} & \text{if } j-w_i \ge 0 \\ V[i-1,j] & \text{if } j-w_i < 0 \end{cases}$$

Initial conditions: $V[0,j] = 0$ and $V[i,0] = 0$

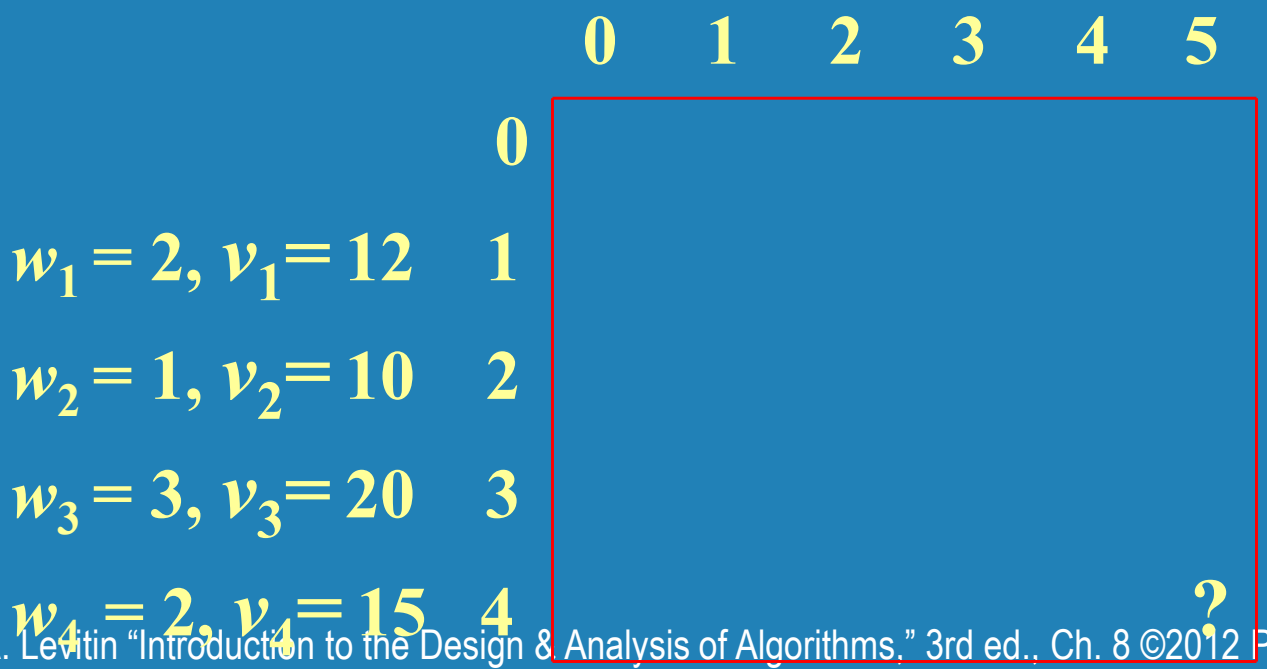


A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Knapsack Problem by DP (example)

Example: Knapsack of capacity $W = 5$

item	weight	value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15



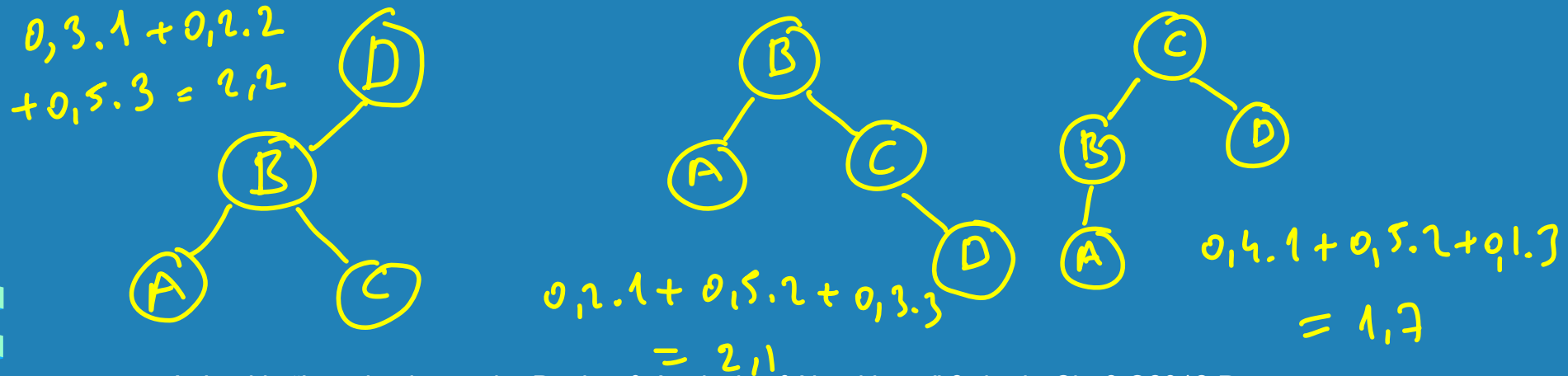
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Optimal Binary Search Trees

Problem: Given n keys $a_1 < \ldots < a_n$ and probabilities $p_1 \leq \ldots \leq p_n$ searching for them, find a BST with a minimum average number of comparisons in successful search.

Since total number of BSTs with n nodes is given by $C(2n,n)/(n+1)$, which grows exponentially, brute force is hopeless.

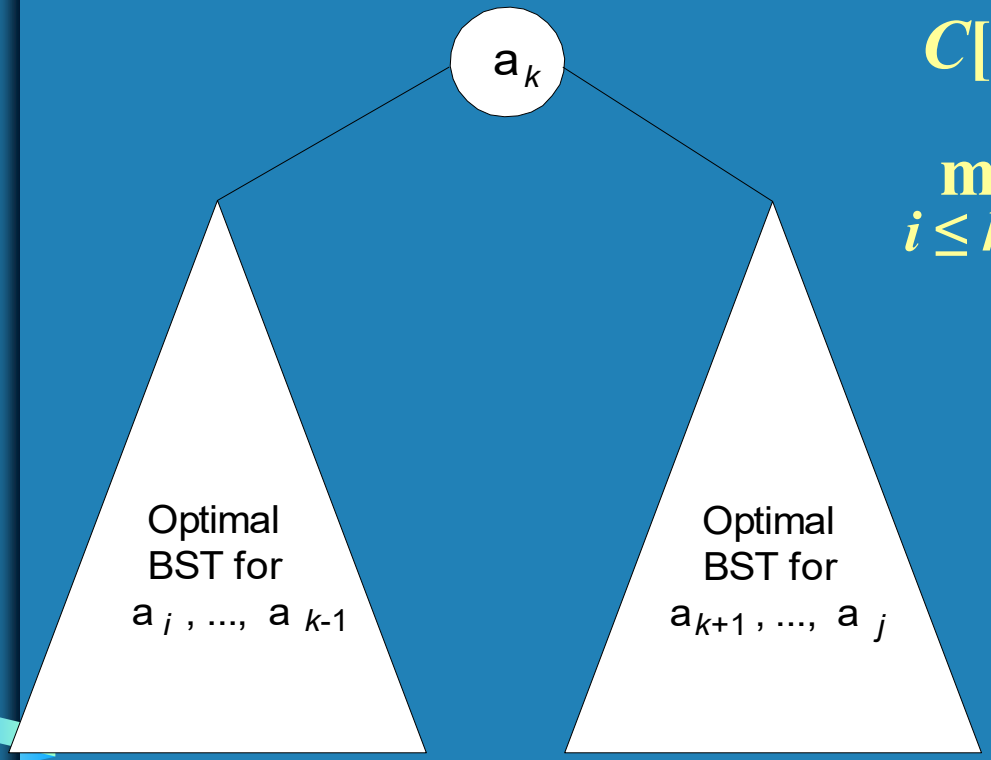
Example: What is an optimal BST for keys A, B, C, and D with search probabilities 0.1, 0.2, 0.4, and 0.3, respectively?



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

DP for Optimal BST Problem

Let $C[i,j]$ be minimum average number of comparisons made in T[i,j], optimal BST for keys $a_i < \ldots < a_j$, where $1 \le i \le j \le n$. Consider optimal BST among all BSTs with some a_k ($i \le k \le j$) as their root; T[i,j] is the best among them.



$$C[i,j] = \min_{i \le k \le j} \{p_k \cdot 1 + \sum_{s=i}^{k-1} p_s \text{ (level } a_s \text{ in T}[i,k-1] +1) + \sum_{s=k+1}^{j} p_s \text{ (level } a_s \text{ in T}[k+1,j] +1)\}$$

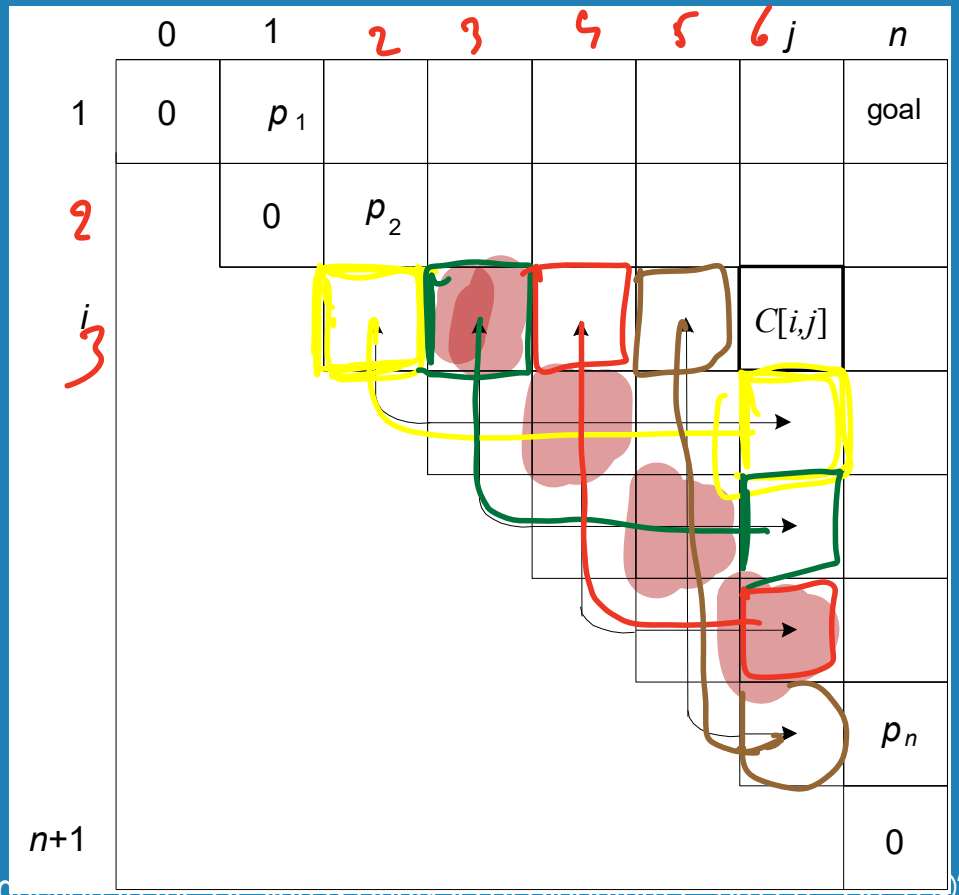
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

DP for Optimal BST Problem (cont.)

After simplifications, we obtain the recurrence for $C[i,j]$:

$$C[i,j] = \min_{i \le k \le j} \{C[i,k-1] + C[k+1,j]\} + \sum_{s=i}^{j} p_s \quad \text{for } 1 \le i \le j \le n$$

$$C[i,i] = p_i \quad \text{for } 1 \le i \le j \le n$$



C[3,6]

k:3 C[3,2] + C[4,6]

k:4 C[3,3] + C[5,6]

k:5 C[3,4] + C[6,6]

k:6 C[3,5] + C[7,6]

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Example: key A B C D

probability 0.1 0.2 0.4 0.3

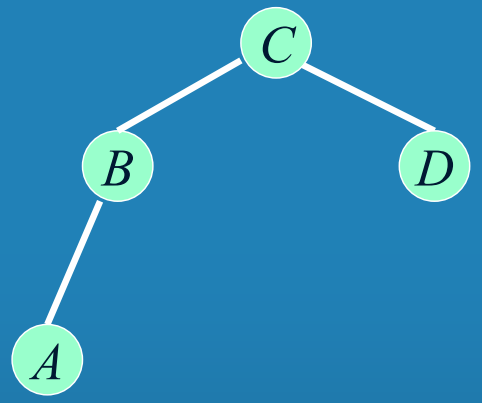
The tables below are filled diagonal by diagonal: the left one is filled using the recurrence

$$C[i,j] = \min_{i \le k \le j} \{C[i,k\text{-}1] + C[k\text{+}1,j]\} + \sum_{s=i}^{j} p_s, \quad C[i,i] = p_i;$$

the right one, for trees' roots, records k's values giving the minima

i \ j	0	1	2	3	4
1	0	.1	.4	1.1	1.7
2		0	.2	.8	1.4
3			0	.4	1.0
4				0	.3
5					0

i \ j	0	1	2	3	4
1		1	2	3	3
2			2	3	3
3				3	3
4					4
5					



optimal BST

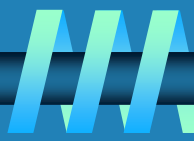
ALGORITHM *OptimalBST*$(P[1..n])$

//Finds an optimal binary search tree by dynamic programming
//Input: An array $P[1..n]$ of search probabilities for a sorted list of n keys
//Output: Average number of comparisons in successful searches in the
// optimal BST and table R of subtrees' roots in the optimal BST

for $i \leftarrow 1$ **to** n **do**
  $C[i, i-1] \leftarrow 0$
  $C[i, i] \leftarrow P[i]$
  $R[i, i] \leftarrow i$
$C[n+1, n] \leftarrow 0$
for $d \leftarrow 1$ **to** $n-1$ **do** //diagonal count
  for $i \leftarrow 1$ **to** $n-d$ **do**
    $j \leftarrow i+d$
    $minval \leftarrow \infty$
    for $k \leftarrow i$ **to** j **do**
      if $C[i, k-1] + C[k+1, j] < minval$
        $minval \leftarrow C[i, k-1] + C[k+1, j]$; $kmin \leftarrow k$
    $R[i, j] \leftarrow kmin$
    $sum \leftarrow P[i]$; **for** $s \leftarrow i+1$ **to** j **do** $sum \leftarrow sum + P[s]$
    $C[i, j] \leftarrow minval + sum$
return $C[1, n]$, R

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

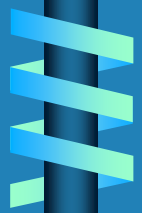
Analysis DP for Optimal BST Problem



Time efficiency: $\Theta(n^3)$ **but can be reduced to** $\Theta(n^2)$ **by taking advantage of monotonicity of entries in the root table, i.e.,** $R[i,j]$ **is always in the range between** $R[i,j-1]$ **and** $R[i+1,j]$

Space efficiency: $\Theta(n^2)$

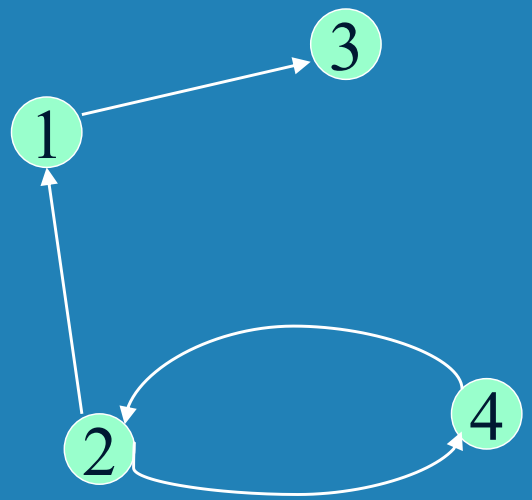
Method can be expended to include unsuccessful searches



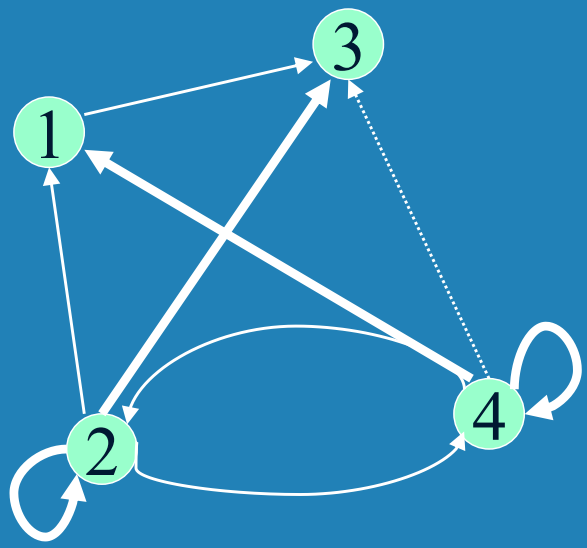
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Warshall's Algorithm: Transitive Closure

- Computes the transitive closure of a relation
- Alternatively: existence of all nontrivial paths in a digraph
- Example of transitive closure:



$$\begin{matrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{matrix}$$

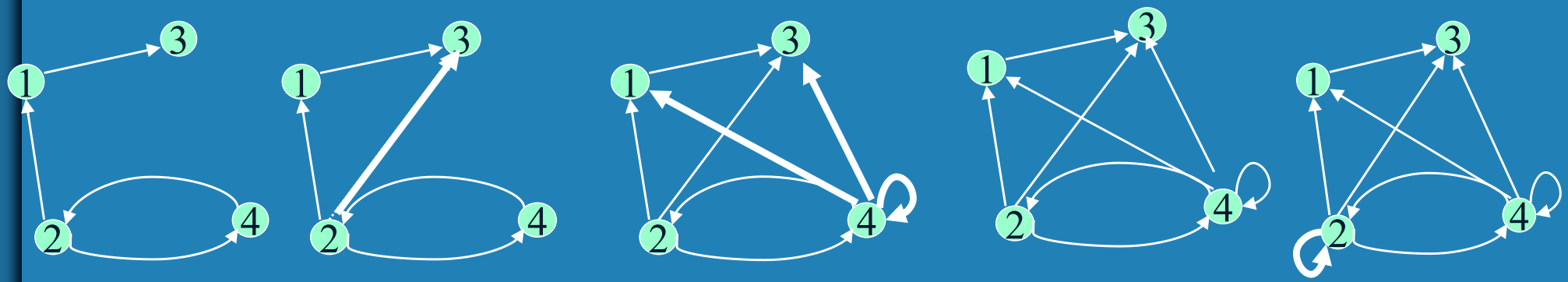


$$\begin{matrix} 0 & 0 & 1 & 0 \\ 1 & \mathbf{1} & \mathbf{1} & 1 \\ 0 & 0 & 0 & 0 \\ \mathbf{1} & 1 & \mathbf{1} & \mathbf{1} \end{matrix}$$

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Warshall's Algorithm

Constructs transitive closure T as the last matrix in the sequence of n-by-n matrices $R^{(0)}, \dots, R^{(k)}, \dots, R^{(n)}$ where
$R^{(k)}[i,j] = 1$ iff there is nontrivial path from i to j with only first k vertices allowed as intermediate
Note that $R^{(0)} = A$ (adjacency matrix), $R^{(n)} = T$ (transitive closure)



$R^{(0)}$

0	0	1	0
1	0	0	1
0	0	0	0
0	1	0	0

$R^{(1)}$

0	0	1	0
1	0	**1**	1
0	0	0	0
0	1	0	0

$R^{(2)}$

0	0	1	0
1	0	1	1
0	0	0	0
1	1	**1**	**1**

$R^{(3)}$

0	0	1	0
1	0	1	1
0	0	0	0
1	1	1	1

$R^{(4)}$

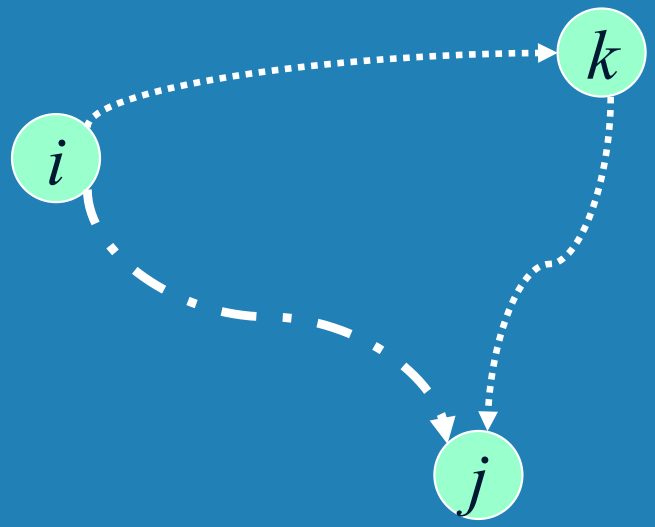
0	0	1	0
1	**1**	1	1
0	0	0	0
1	1	1	1

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Warshall's Algorithm (recurrence)

On the *k*-th iteration, the algorithm determines for every pair of vertices *i, j* if a path exists from *i* and *j* with just vertices 1,…,*k* allowed as intermediate

$$R^{(k)}[i,j] = \begin{cases} R^{(k-1)}[i,j] & \text{(path using just 1 ,…,}k\text{-1)} \\ \quad \text{or} & \\ R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j] & \text{(path from } i \text{ to } k \\ & \quad \text{and from } k \text{ to } i \\ & \quad \text{using just 1 ,…,}k\text{-1)} \end{cases}$$



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Warshall's Algorithm (matrix generation)



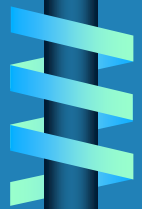
Recurrence relating elements $R^{(k)}$ to elements of $R^{(k-1)}$ is:

$$R^{(k)}[i,j] = R^{(k-1)}[i,j] \text{ or } (R^{(k-1)}[i,k] \text{ and } R^{(k-1)}[k,j])$$

It implies the following rules for generating $R^{(k)}$ from $R^{(k-1)}$:

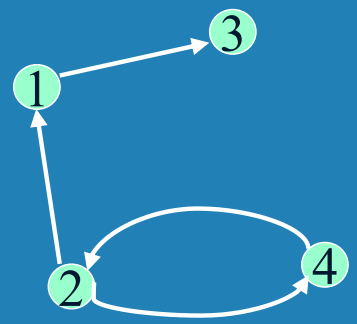
Rule 1 If an element in row i and column j is 1 in $R^{(k-1)}$, it remains 1 in $R^{(k)}$

Rule 2 If an element in row i and column j is 0 in $R^{(k-1)}$, it has to be changed to 1 in $R^{(k)}$ if and only if the element in its row i and column k and the element in its column j and row k are both 1's in $R^{(k-1)}$



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Warshall's Algorithm (example)



$$R^{(0)} = \begin{array}{|cccc|}\hline 0 & 0 & 1 & 0 \\ \hline 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \hline \end{array}$$

$$R^{(1)} = \begin{array}{|c|ccc|}\hline 0 & 0 & 1 & 0 \\ \hline 1 & 0 & \mathbf{1} & 1 \\ \hline 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ \hline \end{array}$$

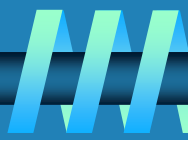
$$R^{(2)} = \begin{array}{|cc|c|c|}\hline 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ \hline 0 & 0 & 0 & 0 \\ \hline \mathbf{1} & 1 & \mathbf{1} & \mathbf{1} \\ \hline \end{array}$$

$$R^{(3)} = \begin{array}{|c|ccc|}\hline 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ \hline 1 & 1 & 1 & 1 \\ \hline \end{array}$$

$$R^{(4)} = \begin{array}{cccc} 0 & 0 & 1 & 0 \\ 1 & \mathbf{1} & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{array}$$

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Warshall's Algorithm (pseudocode and analysis)



ALGORITHM *Warshall*$(A[1..n, 1..n])$

//Implements Warshall's algorithm for computing the transitive closure
//Input: The adjacency matrix A of a digraph with n vertices
//Output: The transitive closure of the digraph
$R^{(0)} \leftarrow A$
for $k \leftarrow 1$ **to** n **do**
    for $i \leftarrow 1$ **to** n **do**
        for $j \leftarrow 1$ **to** n **do**
            $R^{(k)}[i, j] \leftarrow R^{(k-1)}[i, j]$ **or** $(R^{(k-1)}[i, k]$ **and** $R^{(k-1)}[k, j])$
return $R^{(n)}$

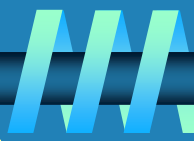
Time efficiency: $\Theta(n^3)$

Space efficiency: Matrices can be written over their predecessors



A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

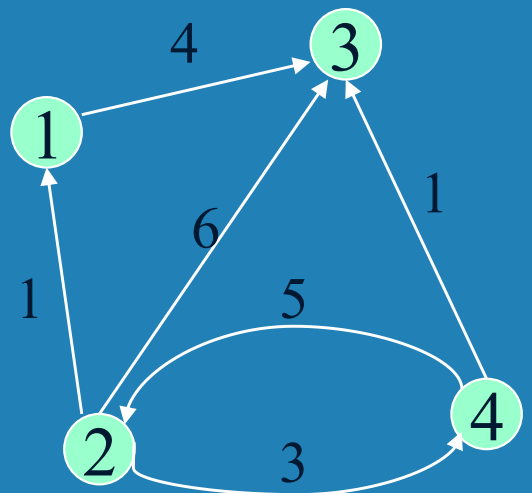
Floyd's Algorithm: All pairs shortest paths



Problem: In a weighted (di)graph, find shortest paths between every pair of vertices

Same idea: construct solution through series of matrices $D^{(0)}$, …, $D^{(n)}$ using increasing subsets of the vertices allowed as intermediate

Example:





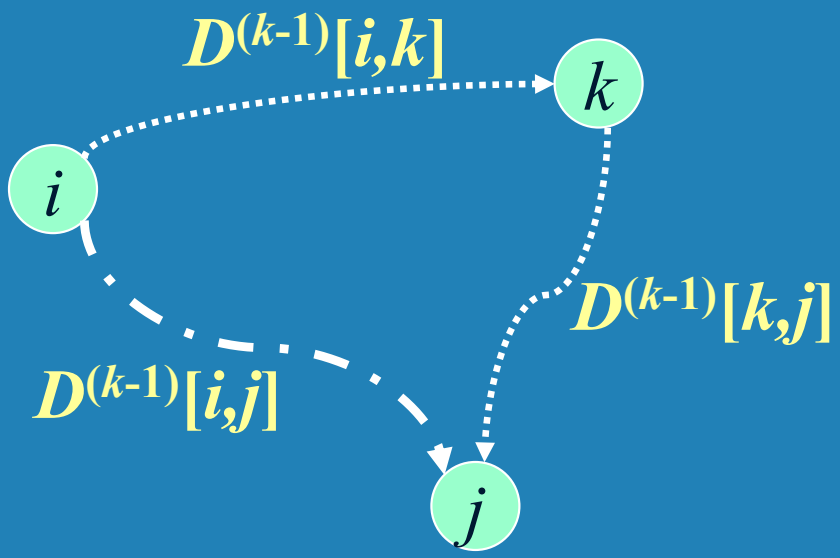
A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Floyd's Algorithm (matrix generation)



On the *k*-th iteration, the algorithm determines shortest paths between every pair of vertices *i, j* that use only vertices among 1,…,*k* as intermediate

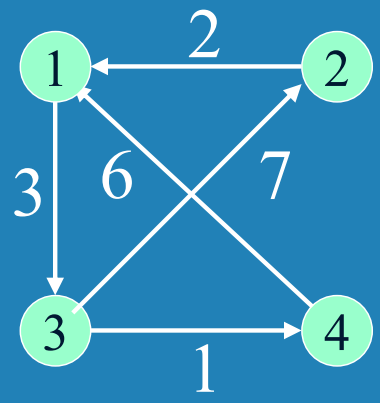
$$D^{(k)}[i,j] = \min \{D^{(k-1)}[i,j],\ D^{(k-1)}[i,k] + D^{(k-1)}[k,j]\}$$





A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Floyd's Algorithm (example)



$D^{(0)} =$

0	∞	3	∞
2	0	∞	∞
∞	7	0	1
6	∞	∞	0

$D^{(1)} =$

0	∞	3	∞
2	0	**5**	∞
∞	7	0	1
6	∞	**9**	0

$D^{(2)} =$

0	∞	3	∞
2	0	5	∞
9	7	0	1
6	∞	9	0

$D^{(3)} =$

0	**10**	3	**4**
2	0	5	**6**
9	7	0	1
6	**16**	9	0

$D^{(4)} =$

0	10	3	4
2	0	5	6
7	7	0	1
6	16	9	0

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.

Floyd's Algorithm (pseudocode and analysis)

ALGORITHM *Floyd*$(W[1..n, 1..n])$

```
//Implements Floyd's algorithm for the all-pairs shortest-paths problem
//Input: The weight matrix W of a graph with no negative-length cycle
//Output: The distance matrix of the shortest paths' lengths
D ← W //is not necessary if W can be overwritten
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            D[i, j] ← min{D[i, j], D[i, k] + D[k, j]}
return D
```

Time efficiency: $\Theta(n^3)$

Space efficiency: Matrices can be written over their predecessors

Note: Shortest paths themselves can be found, too (Problem 10)

A. Levitin "Introduction to the Design & Analysis of Algorithms," 3rd ed., Ch. 8 ©2012 Pearson Education, Inc. Upper Saddle River, NJ. All Rights Reserved.